

✓ Prática: Redes Neurais Convolucionais

Vamos agora implementar a rede [AlexNet](#), uma das redes que trouxeram todo esse interesse para a área de *deep learning*.

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
```

```
from torch import optim
from torchsummary import summary
```

```
import time, os, sys, numpy as np
```

```
# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids= list(range(n))
```

```
def load_data_cifar10(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Cifar10-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)
```

```
mnist_train = torchvision.datasets.CIFAR10(root=root, train=True,download=True,transform=transformer)
mnist_test = torchvision.datasets.CIFAR10(root=root, train=False,download=True,transform=transformer)
num_workers = 0 if sys.platform.startswith('win32') else 4
```

```
train_iter = torch.utils.data.DataLoader(mnist_train,
                                          batch_size, shuffle=True,
                                          num_workers=num_workers)
test_iter = torch.utils.data.DataLoader(mnist_test,
                                         batch_size, shuffle=False,
                                         num_workers=num_workers)

return train_iter, test_iter
```

funções básicas

```
def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])
```

Função usada para calcular acurácia

```
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""
```

```
    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]
```

```
    return acc_sum.item() / n, l.item() / len(data_iter)
```

Função usada no treinamento e validação da rede

```
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
    print('training on', device)
    for epoch in range(num_epochs):
        net.train()
        train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
        for X, y in train_iter:
```

```

X, y = X.to(device), y.to(device)
y_hat = net(X)
trainer.zero_grad()
l = loss(y_hat, y).sum()
l.backward()
trainer.step()
train_l_sum += l.item()
train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
n += y.size()[0]
test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
      'test acc %.3f, time %.1f sec'
      % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
         test_acc, time.time() - start))

```

✓ AlexNet

Agora já temos todo o conhecimento necessário para implementar nossa primeira arquitetura moderna. Vamos implementar a [AlexNet](#), uma das arquiteturas mais famosas dessa nova onda de rede neurais.



AlexNet Network - Structural Details													
Input			Output			Layer	Stride	Pad	Kernel size		in	out	# of Param
227	227	3	55	55	96	conv1	4	0	11	11	3	96	34944
55	55	96	27	27	96	maxpool1	2	0	3	3	96	96	0
27	27	96	27	27	256	conv2	1	2	5	5	96	256	614656
27	27	256	13	13	256	maxpool2	2	0	3	3	256	256	0
13	13	256	13	13	384	conv3	1	1	3	3	256	384	885120
13	13	384	13	13	384	conv4	1	1	3	3	384	384	1327488
13	13	384	13	13	256	conv5	1	1	3	3	384	256	884992
13	13	256	6	6	256	maxpool5	2	0	3	3	256	256	0
						fc6			1	1	9216	4096	37752832
						fc7			1	1	4096	4096	16781312
						fc8			1	1	4096	1000	4097000
						Total							62,378,344

Lembre-se que, após cada camada de convolução e linear, há uma ativação não linear ReLU.

```

# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 20, 0.001, 100, 0.0001

# Implementa sua rede neural aqui
# Você pode implementar criando uma variável net e adicionando as camadas net = nn.Sequential()
# Ou pode criar como uma classe(Preferido, pois fica mais organizado e é mais facilitado para um debug da rede)
# rede baseada na AlexNet
class AlexNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):
        super(AlexNet, self).__init__(**kwargs)
        self.convs = nn.Sequential(
            nn.Conv2d(in_channels=input_channels, out_channels=96, kernel_size=11, stride=4, padding=0), # entrada: (b, 3, 227, 227) e saída: (b, 96, 55, 55)
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0), # entrada: (b, 96, 55, 55) e saída: (b, 96, 27, 27)

            nn.Conv2d(in_channels=96, out_channels=256, kernel_size=5, stride=1, padding=2), # entrada: (b, 96, 27, 27) e saída: (b, 256, 27, 27)
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0), # entrada: (b, 256, 27, 27) e saída: (b, 256, 13, 13)
            nn.Conv2d(in_channels=256, out_channels=384, kernel_size=3, stride=1, padding=1), # entrada: (b, 256, 13, 13) e saída: (b, 384, 13, 13)
            nn.ReLU(),
            nn.Conv2d(in_channels=384, out_channels=384, kernel_size=3, stride=1, padding=1), # entrada: (b, 384, 13, 13) e saída: (b, 384, 13, 13)
            nn.ReLU(),
            nn.Conv2d(in_channels=384, out_channels=256, kernel_size=3, stride=1, padding=1), # entrada: (b, 384, 13, 13) e saída: (b, 256, 13, 13)
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0) # entrada: (b, 256, 13, 13) e saída: (b, 256, 6, 6)
        )

        self.features = nn.Sequential(
            nn.Flatten(), # entrada: (b, 256, 6, 6) e saída: (b, 15552)
            nn.Linear(9216, 4096), # entrada: (b, 15552) e saída: (b, 4096)
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096), # entrada: (b, 4096) e saída: (b, 4096)
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(4096, classes) # entrada: (b, 4096) e saída: (b, classes)
        )

    def forward(self, x):

```

```

def forward(self, x):
    x = self.convs(x)
    x = self.features(x)
    return x

net = AlexNet(3, 10)

# Sending model to device
net.to(device)
print(summary(net,(3,227,227))) # visualize number of parameters' net, output of each layer and total mega bytes necessary f
# and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# trainer do gluon
trainer = optim.Adam(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)

```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 96, 55, 55]	34,944
ReLU-2	[-1, 96, 55, 55]	0
MaxPool2d-3	[-1, 96, 27, 27]	0
Conv2d-4	[-1, 256, 27, 27]	614,656
ReLU-5	[-1, 256, 27, 27]	0
MaxPool2d-6	[-1, 256, 13, 13]	0
Conv2d-7	[-1, 384, 13, 13]	885,120
ReLU-8	[-1, 384, 13, 13]	0
Conv2d-9	[-1, 384, 13, 13]	1,327,488
ReLU-10	[-1, 384, 13, 13]	0
Conv2d-11	[-1, 256, 13, 13]	884,992
ReLU-12	[-1, 256, 13, 13]	0
MaxPool2d-13	[-1, 256, 6, 6]	0
Flatten-14	[-1, 9216]	0
Linear-15	[-1, 4096]	37,752,832
ReLU-16	[-1, 4096]	0
Dropout-17	[-1, 4096]	0
Linear-18	[-1, 4096]	16,781,312
ReLU-19	[-1, 4096]	0
Dropout-20	[-1, 4096]	0
Linear-21	[-1, 10]	40,970

```

=====
Total params: 58,322,314
Trainable params: 58,322,314
Non-trainable params: 0

```

```

-----
Input size (MB): 0.59
Forward/backward pass size (MB): 11.11
Params size (MB): 222.48
Estimated Total Size (MB): 234.18

```

```

None
Files already downloaded and verified
Files already downloaded and verified
training on cuda
epoch 1, train loss 1.9252, train acc 0.280, test loss 1.5785, test acc 0.422, time 145.4 sec
epoch 2, train loss 1.5350, train acc 0.438, test loss 1.3986, test acc 0.494, time 145.4 sec
epoch 3, train loss 1.3851, train acc 0.499, test loss 1.3487, test acc 0.518, time 145.6 sec
epoch 4, train loss 1.2814, train acc 0.542, test loss 1.2514, test acc 0.561, time 146.0 sec
epoch 5, train loss 1.1985, train acc 0.573, test loss 1.1837, test acc 0.580, time 146.5 sec
epoch 6, train loss 1.1084, train acc 0.606, test loss 1.1107, test acc 0.613, time 146.3 sec
epoch 7, train loss 1.0513, train acc 0.626, test loss 1.0765, test acc 0.623, time 146.3 sec
epoch 8, train loss 0.9911, train acc 0.650, test loss 1.0155, test acc 0.642, time 146.8 sec
epoch 9, train loss 0.9408, train acc 0.666, test loss 0.9945, test acc 0.652, time 146.6 sec
epoch 10, train loss 0.8951, train acc 0.684, test loss 0.9723, test acc 0.653, time 145.6 sec
epoch 11, train loss 0.8508, train acc 0.699, test loss 0.9437, test acc 0.674, time 146.1 sec
epoch 12, train loss 0.8150, train acc 0.712, test loss 0.9237, test acc 0.675, time 146.2 sec
epoch 13, train loss 0.7816, train acc 0.724, test loss 0.9265, test acc 0.676, time 145.1 sec
epoch 14, train loss 0.7487, train acc 0.734, test loss 0.9479, test acc 0.671, time 146.0 sec
epoch 15, train loss 0.7177, train acc 0.747, test loss 0.9342, test acc 0.678, time 145.4 sec
epoch 16, train loss 0.6931, train acc 0.752, test loss 0.9367, test acc 0.687, time 146.1 sec
epoch 17, train loss 0.6631, train acc 0.766, test loss 0.9109, test acc 0.695, time 146.9 sec
epoch 18, train loss 0.6430, train acc 0.771, test loss 0.9234, test acc 0.694, time 144.9 sec
epoch 19, train loss 0.6184, train acc 0.783, test loss 0.9064, test acc 0.689, time 144.5 sec
epoch 20, train loss 0.5978, train acc 0.788, test loss 0.9298, test acc 0.698, time 145.9 sec

```

